Ukonnen's Algorithm:

It is the chosen algorithm for the construction of the Suffix Tree and it does so in O(n) time / space complexity; this report assumes that this is given information and will not go into the specifics of explaining how it was obtained.

In terms of algorithm design, I chose to use the "active point" approach for handling extensions in the data structure, which makes use of active edge and active length besides the traditional approach of using active node only, and that was inspired upon reading the Stack Overflow post referenced below.

Reference:
Ukkonen's suffix tree algorithm in plain English. Stack Overflow. (2012, February 26).
https://stackoverflow.com/questions/9452701/ukkonens-suffix-tree-algorithm-in-plain-english

LCP Computation:

My approach for solving this problem starts with performing a lexicographic depth first search in order to find all the leaves in the suffix tree in the desired order. A leaf node is distinguished from other internal nodes based on its leaf number and the complexity analysis of this step is [Time: O(n log |∑|), Space: O(n)].

Then for every pair of leaves in lexicographic order, their ancestors are compared in order to find the lowest common ancestor which represents the longest common prefix in this context. Each node contains a pointer / reference to its parent which is used to traverse that path all the way to the root, and then a check is performed in reverse order, in order to find the deepest match. Lastly, once the ancestor is found, we can simply return its depth which was assigned to it during its creation. The complexity analysis of this step is [Time: O(n * d), Space: O(n)]

Hence the overall complexity of this task is O(n *d) / O(n^2) for time complexity and O(n) for space complexity. The n squared was included since in the worst case depth is n.

Symbols used:

- *n is the length of the input string.*
- *d is the maximum node depth in the suffix tree, equivalent to the number of characters from the root.*
- |∑| *is the size of the alphabet of the input string.*